

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Maestría en Ciencias de la Computación

Inteligencia artificial

Profesor: Doctor Iván Olmos Pineda

3.º Parcial

Red neuronal artificial (RNA) con ajuste de
hiperparámetros (GA + GWO)

Alumno: José Daniel Flores Morales

Matrícula: 225470536

Especialidad: Sistemas distribuidos

Instrucciones: Analiza cada una de las preguntas que se exponen a continuación. Realiza las implementaciones correspondientes, las cuales se deberán entregar en la fecha indicada.

Implementar el entrenamiento de una RNA basada en Scikit-learn con Python para una tarea de aprendizaje supervisado usando un clasificador para clases discretas, haciendo un ajuste de hiperparámetros a partir de un algoritmo metaheurístico:

1. (1.0 pts) Busca y selecciona una base de datos que se ajuste a la tarea planteada. La base de datos debe ser tabular, numérica continua y debe tener por lo menos un conjunto de 10 descriptores (o más) y un mínimo de tres clases. Se recomienda que la cantidad de instancias tenga como mínimo 5000 registros. Se debe documentar el origen del dataset, para qué se creó, así como información adicional que ayude a entender qué se busca resolver con el dataset.
2. (2.0 pts) Definir una propuesta que permita ajustar hiperparámetros de una red neuronal artificial a partir de la implementación de un algoritmo metaheurístico híbrido, donde se tome como base un algoritmo genético en combinación con el algoritmo ACO o GWO. Definir la propuesta a través de pseudocódigo Argumentar respuesta.
3. (2.0 pts) Implementar la propuesta definida en el paso 2 usando el lenguaje Python con SkLearn con el módulo MLP. Para realizar el proceso de entrenamiento, usar un proceso de construcción de la muestra aleatorio para el train y el test. Se debe cuidar que la cantidad de instancias, tanto en el train como en el test, no quede con fuertes desbalances.
4. (3.0 pts) Documentar estadísticamente la evolución de los resultados en el proceso de ajuste de hiperparámetros. Se recomienda usar gráficas que definan aspectos clave como la evolución de los resultados por iteraciones (GWO), el tamaño de la población para el algoritmo genético, comportamiento estadístico de tendencia central de todos los resultados que van obteniendo por iteración. Realizar un análisis de resultados obtenidos.
5. (2.0 pts) Crear un video y compartir el link correspondiente, de una longitud no mayor de 5 minutos, en el cual se explique el proyecto y se presenten pruebas de ejecución (NOTA: verificar que el link es accesible, ya que, de no tener acceso, se asignará 0 puntos para esta pregunta).

Fecha de entrega: la señalada en clase.

Entregables:

1. Documento PDF con respuestas a cada una de las preguntas planteadas.
2. Archivo comprimido (.zip, .rar) con los códigos correspondientes (basado en el proyecto compartido por el profesor).

Pregunta No. Uno: Selección del dataset

Dry Bean Dataset (frijol seco)

Clasificación multiclase de frijoles secos mediante técnicas de visión artificial y aprendizaje automático.

Resumen

Se desarrolló un sistema de visión artificial para distinguir variedades registradas diferentes de frijoles secos con características similares, con el fin de obtener una clasificación uniforme de las semillas y, con estos resultados, se satisfacen en gran medida las demandas de los productores en cuanto a la obtención de variedades de frijoles uniformes.

En resumen, existe una amplia diversidad genética en el frijol seco, que es el cultivo de leguminosas comestibles más producido en el mundo. La calidad de la semilla influye considerablemente en la producción agrícola. Por lo tanto, la clasificación de semillas es esencial tanto para la comercialización como para la producción, ya que proporciona los principios de los sistemas agrícolas sostenibles. **El objetivo principal de este estudio es proporcionar un método para obtener variedades de semillas uniformes a partir de la producción agrícola, que se realiza en forma de población, de modo que las semillas no estén certificadas como una sola variedad.** Así, se desarrolló un sistema de visión artificial para distinguir siete variedades registradas diferentes de frijol seco con características similares, con el fin de obtener una clasificación uniforme de las semillas. **Para el modelo de clasificación, se tomaron imágenes de 13,611 granos de 7 variedades registradas diferentes de frijol seco con una cámara de alta resolución.** Se diseñó una interfaz de usuario intuitiva utilizando la interfaz gráfica de usuario (GUI) de MATLAB. Las imágenes de los granos obtenidas por el sistema de visión artificial (CVS) se sometieron a etapas de segmentación y extracción de características, y **se obtuvieron un total de 16 características: 12 dimensionales y 4 de forma, a partir de los granos.** Se crearon modelos de clasificación de perceptrón multicapa (MLP), máquina de vectores de soporte (SVM), k-vecinos más cercanos (kNN) y árbol de decisión (DT) con validación cruzada de 10 pliegues y se compararon las métricas de rendimiento. Las tasas de clasificación correctas generales se determinaron en 91,73 %, 93,13 %, 87,92 % y 92,52 % para MLP, SVM, kNN y DT, respectivamente. El modelo de clasificación SVM, que obtuvo los resultados de mayor precisión, clasificó las variedades de frijol Barbunya, Bombay, Cali, Dermason, Horoz, Seker y Sira con 92,36 %, 100,00 %, 95,03 %, 94,36 %, 94,92 %, 94,67 % y 86,84 %, respectivamente. Con estos resultados, se satisfacen en gran medida las demandas de los productores y los clientes en cuanto a la obtención de variedades de frijol uniformes.

<https://www.semanticscholar.org/paper/Dry-Beans-Classification-using-Ensemble-Learning-Shriya-Kumar/9fd25afc267d889b7b9fdc5fadaec0dbdde98cf6>

Origen y fuente

El dataset fue creado por Murat Koklu e Ilker Ali Ozkan, investigadores de la Universidad de Selcuk en Turquía, y publicado en el repositorio oficial de la UCI Machine Learning Repository en 2020.

- **Fuente:** UCI Machine Learning Repository
- **URL:** <https://archive.ics.uci.edu/dataset/602/dry+bean+dataset>
- **Referencia:** Koklu, M. & Ozkan, I.A. (2020). Multiclass classification of dry beans using computer vision and machine learning techniques. Computers and Electronics in Agriculture, 174.

¿Para qué se creó?

El objetivo del dataset es automatizar la clasificación visual de variedades de frijol seco mediante visión por computadora y machine learning. Tradicionalmente, la clasificación de semillas se hace de forma manual por expertos, un proceso lento, costoso y sujeto a error humano. Este trabajo buscó demostrar que es posible clasificar automáticamente las variedades a partir de características morfológicas extraídas con procesamiento de imágenes.

En términos prácticos, tiene aplicaciones discretas en la industria alimentaria y agropecuaria: control de calidad en emparadoras, selección automatizada para entrenarse para siembra y certificación de variedades comerciales.

¿Qué se busca resolver?

El problema es de **clasificación multiclase supervisada**: dado el conjunto de características geométricas y morfológicas de un frijol, determina a cuál de las 7 variedades pertenece. Es un clasificador para clases discretas con features numéricas continuas, lo que lo hace ideal para entenderse con una red neuronal tipo MLP.

Características del dataset

Propiedad	Valor
Número de instancias	13,611
Número de features (descriptores)	16
Número de clases	7
Tipo de features	Numéricas continuas
Valores nulos	Ninguno
Tipo de tareas	Clasificación multiclase supervisada

Las 7 clases (variantes de frijol)

Clase	Nombre común	Características distintivas
SEKER	Frijol azúcar	Forma redonda, superficie lisa, tamaño mediano
BARBUNYA	Frijol pinto turco	Manchas características, forma ovalada
BOMBAY	Frijol bombay	El más grande de los 7, alta área y perímetro
CALI	Frijol cali	Forma alargada, eccentricidad alta
HOROZ	Frijol horoz	Forma muy alargada, aspecto ratio elevado
SIRA	Frijol sira	Color amarillento, forma intermedia
DERMASON	Frijol dermason	El más pequeño, más abundante en el dataset

Descripción de los 16 descriptores (features)

Todos los valores son numéricos continuos extraídos de imágenes digitales de los frijoles mediante procesamiento de imágenes.

#	Feature	Descripción
1	Área	Área de la región del frijol en píxeles
2	Perímetro	Perímetro del contorno del frijol en píxeles
3	MajorAxisLength	Longitud del eje mayor de la elipse equivalente
4	MinorAxisLength	Longitud del eje menor de la elipse equivalente
5	AspectRation	Relación entre eje mayor y eje menor (forma)
6	Eccentricity	¿Qué tan alargada es la forma (0=círculo, 1=línea)?
7	ConvexArea	Área del casco convexo que envuelve al frijol
8	EquivDiameter	Diámetro de un círculo con la misma área que el frijol
9	Extent	Proporción del área del frijol respecto a su bounding box
10	Solidity	Proporción del área respecto al casco convexo
11	roundness	¿Qué tan circular es la forma del frijol?
12	Compactness	Relación entre el área y el perímetro al cuadrado
13	ShapeFactor1	Relación entre eje mayor y área
14	ShapeFactor2	Relación entre eje menor y área
15	ShapeFactor3	Compacidad al cuadrado
16	ShapeFactor4	Convexidad normalizada

Análisis Exploratorio de Datos EDA – analisis_exploratorio.py

Propósito general

Este módulo implementa el Análisis Exploratorio de Datos (EDA) sobre el Dry Bean Dataset. Su función dentro del proyecto es estrictamente de diagnóstico previo: se ejecuta una sola vez antes de diseñar el pipeline de entrenamiento, con el objetivo de entender la estructura del dataset, detectar problemas de calidad y justificar las decisiones del preprocesado. **No forma parte del flujo de entrenamiento del modelo.**

El módulo genera cuatro gráficas exportadas en “**Gráficas/EDA**” y manda información del dataset a consola que documenta el estado real del dataset.

Función cargar_datos(ruta)

Lee el archivo CSV del dataset y lo retorna como un DataFrame de pandas sin ninguna transformación.

```
def cargar_datos(ruta):  
    df = pd.read_csv(ruta)  
    return df
```

Carga el archivo en memoria. En este punto, el dataset contiene 13,611 filas y 17 columnas (16 features + columna Class). No se eliminan duplicados aquí; eso es responsabilidad de preprocessing.py para que el EDA refleje el estado crudo real del dataset.

Función informacion_general(df)

Realiza un diagnóstico completo de calidad del dataset antes de cualquier transformación.

```
def informacion_general(df):  
    print(df.dtypes)  
  
    nulos = df.isnull().sum()  
    print(nulos)  
    print(f"Total de celdas nulas: {nulos.sum()}")  
  
    duplicados = df.duplicated().sum()  
    print(f"Total de filas duplicadas: {duplicados}")  
  
    print(df.describe())  
  
print(df.describe())
```

Línea	¿Qué revela?
-------	--------------

<code>df.dtypes</code>	Detecta si alguna feature numérica fue interpretada como <code>object</code> por error de formato.
<code>df.isnull().sum()</code>	Identifica columnas con valores faltantes que requerirían imputación.
<code>df.duplicated().sum()</code>	Cuenta filas repetidas exactas —el dataset tiene 68 duplicados que se eliminan en el preprocesado.
<code>df.describe()</code>	Expone media, desviación estándar, mínimo y máximo de cada feature, permitiendo detectar outliers evidentes y comparar escalas entre variables.

Resultado obtenido: 0 valores nulos en todas las columnas, 68 filas duplicadas. Las features tienen rangos muy dispares (Area va de 20,420 a 254,616, mientras Eccentricity va de 0.30 a 0.99), lo que justifica el uso de **StandardScaler** en el preprocesado.

Función `distribucion_clases(df)`

Analiza el balance entre las 7 variedades de frijol y genera un gráfico de barras con el conteo por clase.

```
def distribucion_clases(df):
    conteo = df["Class"].value_counts()

    for clase, n in conteo.items():
        porcentaje = n / len(df) * 100
        print(f" {clase:<12} {n:>5} muestras  ({porcentaje:.1f}%)")

    conteo.plot(kind="bar", ax=ax, color="steelblue", edgecolor="white")
```

```
conteo = df["Class"].value_counts()
porcentaje = n / len(df) * 100
```

`value_counts()` ordena las clases de mayor a menor frecuencia. El cálculo del porcentaje permite identificar si existe desbalance significativo que sesgaría el clasificador.

Resultado obtenido

Variedad	Muestras	Porcentaje
DERMASON	3,546	26.1%
SIRA	2,636	19.4%
SEKER	2,027	14.9%

HOROZ	1,928	14.2%
CALI	1,630	12.0%
BARBUNYA	1,322	9.7%
BOMBAY	522	3.8%

Función `histogramas_features(df)`

Genera una cuadrícula 4x4 con la distribución de frecuencias de cada una de las 16 features numéricas.

```
def histogramas_features(df):
    features = [c for c in df.columns if c != "Class"]

    cols = 4
    rows = -(-len(features) // cols) # ceil sin importar math
    fig, axes = plt.subplots(rows, cols, figsize=(16, 12))

    for i, col in enumerate(features):
        axes[i].hist(df[col], bins=40, color="steelblue", edgecolor="white")
```

Líneas clave

```
features = [c for c in df.columns if c != "Class"]
axes[i].hist(df[col], bins=40, ...)
```

La exclusión de Class mediante comprensión de lista garantiza que solo se grafiquen variables numéricas. El uso de 40 bins por histograma proporciona suficiente resolución para identificar si las distribuciones son simétricas, sesgadas o bimodales.

¿Para **qué sirve en la práctica**? Si una feature muestra distribución muy sesgada o con valores acumulados en los extremos, puede requerir transformación logarítmica además de la estandarización estándar. Esta función también confirma que todas las features son continuas o enteras, validando que el StandardScaler es el preprocesador apropiado.

Función `heatmap_correlacion(df)`

Calcula la matriz de correlación de Pearson entre todas las features y la visualiza como un mapa de calor. Adicionalmente, imprime en consola todos los pares cuya relación absoluta supera 0.95.


```
def heatmap_correlacion(df):
    features = [c for c in df.columns if c != "Class"]
    correlacion = df[features].corr()

    sns.heatmap(correlacion, annot=True, fmt=".2f", cmap="coolwarm", center=0)

    for i in range(len(correlacion.columns)):
        for j in range(i + 1, len(correlacion.columns)):
            r = correlacion.iloc[i, j]
            if abs(r) > 0.95:
                print(f" {correlacion.columns[i]} <-> {correlacion.columns[j]}: {r:.4f}")
```

Líneas clave

```
correlacion = df[features].corr()
if abs(r) > 0.95:
```

corr() calcula el coeficiente de Pearson entre cada par de columnas, produciendo una matriz simétrica de 16×16. El umbral $|r| > 0.95$ identifica pares con multicolinealidad severa, donde una feature aporta información casi redundante respecto a la otra.

Resultado obtenido — pares con correlación alta:

Par	Correlación
Area ↔ ConvexArea	0.9999
Compactness ↔ ShapeFactor3	0.9987
AspectRatio ↔ Compactness	-0.9877
Perimeter ↔ EquivDiameter	0.9914
Eccentricity ↔ ShapeFactor3	-0.9811
AspectRatio ↔ ShapeFactor3	-0.9786

El par Area ↔ ConvexArea con $r = 0.9999$ es prácticamente una relación lineal perfecta. El par Compactness ↔ ShapeFactor3 con $r = 0.9987$ revela que ShapeFactor3 es una derivación directa de Compactness. Estas redundancias justifican una posible reducción de dimensionalidad, aunque el MLP de scikit-learn puede manejarlo mediante regularización L2 (alpha).

Función boxplots_por_clase(df)

Genera boxplots de cada una de las features por variedad de frijol, permitiendo visualizar el poder discriminativo de cada variable.

```
def boxplots_por_clase(df):
    features = [c for c in df.columns if c != "Class"]
```

```
for i, col in enumerate(features):  
    df.boxplot(column=col, by="Class", ax=axes[i], grid=False)
```

Línea clave

```
df.boxplot(column=col, by="Class", ax=axes[i], grid=False)
```

`boxplot(..., by="Class")` agrupa los valores de cada feature según la variedad, mostrando la mediana, los cuartiles y los valores atípicos de cada clase de forma independiente. Cuando los bigotes de distintas clases se solapan significativamente, esa feature tiene bajo poder discriminativo por sí sola. Cuando las cajas están bien separadas, la feature contribuye a la separabilidad de clases.

¿Para qué sirve en la práctica? Valida que el conjunto de features tiene capacidad de separar las 7 variedades. Si ninguna feature mostrara separación, el problema de clasificación sería intrínsecamente difícil independientemente del modelo. En el Dry Bean Dataset, features como `Area` y `MajorAxisLength` muestran separación notable entre variedades pequeñas (BOMBAY, SEKER) y grandes (DERMASON, CALI).

Flujo de la ejecución

```
python src/analisis_exploratorio.py  
|  
├─ cargar_datos() → DataFrame crudo (13,611 filas)  
├─ informacion_general() → consola: tipos, nulos, duplicados, estadísticas  
├─ distribucion_clases() → Graficas/EDA/distribucion_clases.png  
├─ histogramas_features() → Graficas/EDA/histogramas_features.png  
├─ heatmap_correlacion() → Graficas/EDA/heatmap_correlacion.png  
consola: pares con  $|r| > 0.95$   
├─ boxplots_por_clase() → Graficas/EDA/boxplots_por_clase.png
```

Este módulo no es importado por `main.py`. Su ejecución manual, previa al desarrollo del pipeline, y sus resultados informan decisiones de diseño como la elección de `StandardScaler`, la estratificación del split y el espacio de búsqueda de hiperparámetros del optimizador.

Preprocesado y división de datos: `preprocessing.py` y `split_data.py`

Módulo `preprocessing.py`

Este módulo es el primer eslabón del pipeline de entrenamiento. Su responsabilidad es convertir el CSV crudo del Dry Bean Dataset en estructuras numéricas listas para ser consumidas por cualquier modelo de scikit-learn. Realiza tres transformaciones en secuencia: eliminación de duplicados, estandarización de features y codificación de etiquetas.

Es importado por el main.py y ejecutado una sola vez al inicio del pipeline, antes de la división de datos.

Constante FEATURES

```
FEATURES = [
    "Area", "Perimeter", "MajorAxisLength", "MinorAxisLength",
    "AspectRatio", "Eccentricity", "ConvexArea", "EquivDiameter",
    "Extent", "Solidity", "roundness", "Compactness",
    "ShapeFactor1", "ShapeFactor2", "ShapeFactor3", "ShapeFactor4",
]
```

Define explícitamente las 16 columnas morfológicas que se usan como variables de entrada. La columna Class se excluye aquí ya que es la variable objetivo y se maneja por separado. Tener las features en una constante centralizada permite modificar la selección de variables de un solo lugar sin tocar la lógica de las funciones.

Función cargar_datos()

```
def cargar_datos():
    """Lee el CSV y elimina filas duplicadas."""
    df = pd.read_csv(RUTA_CSV)
    df = df.drop_duplicates()
    print(f"[INFO] Dataset cargado: {df.shape[0]} filas, {df.shape[1]} columnas")
    return df
```

Líneas clave

Línea	Qué hace
pd.read_csv(RUTA_CSV)	Lee el CSV usando la ruta absoluta anclada a __file__, funciona independientemente del directorio desde el que se ejecute el script
df.drop_duplicates()	Elimina las 68 filas duplicadas identificadas en el EDA

Resultado obtenido: El dataset pasa de 13,611 filas a 13,543 filas tras eliminar duplicados. Las 17 columnas se mantienen intactas.

Función preprocesar(df)

```
def preprocesar(df):
    """
    Separa X e y, normaliza las features y codifica las etiquetas.
```

```

Returns:
X          (ndarray):          features          normalizadas.
y          (ndarray):          etiquetas          codificadas          como          enteros.
scaler     (StandardScaler):   para          transformar          datos          nuevos.
le         (LabelEncoder):     para          decodificar          predicciones.
"""

X_raw      = df[FEATURES].values
y_raw      = df["Class"].values

scaler     = StandardScaler()
X          = scaler.fit_transform(X_raw)

le         = LabelEncoder()
y          = le.fit_transform(y_raw)

return X, y, scaler, le

```

La función realiza tres operaciones en secuencia:

1. Separación de features y etiquetas

```

X_raw      = df[FEATURES].values          # shape: (13543, 16)
y_raw      = df["Class"].values          # shape: (13543,)

```

Se extraen dos arreglos NumPy independientes: la matriz de features `X_raw` y el vector de etiquetas `y_raw`. En este punto `y_raw` contiene strings ("BOMBAY", "CALI", "SEKER", "BARBUNYA", "DERMASON", "HOROS", "SIRA").

2. Estandarización con StandardScaler

```

scaler     = StandardScaler()
X          = scaler.fit_transform(X_raw)

```

Aplica la transformación z-score a cada feature de forma independiente:

$$z = (x - \mu) / \sigma$$

Donde μ es la media y σ la desviación estándar de cada columna calculadas sobre el conjunto de entrenamiento. El resultado es que cada feature queda con media 0 y desviación estándar 1, independientemente de su escala original.

Esta transformación es necesaria porque el EDA reveló rangos muy dispares entre features: El área opera en el rango [20,420 – 254,616], mientras que Eccentricity opera [0.30 – 0.99]. Sin estandarizar, el gradiente durante el backpropagation del MLP quedaría dominado por las variables de mayor escala, degradando la convergencia. El objeto `scaler` se retorna porque almacena los parámetros μ y σ calculados. Estos

mismos parámetros deben aplicarse a los conjuntos de validación y test usando únicamente `transform()` — nunca `fit_transform()` de nuevo — para evitar fuga de información entre particiones.

3. Codificación de etiquetas con LabelEncoder

```
le = LabelEncoder()  
y = le.fit_transform(y_raw)
```

Convierte los nombres de las clases en enteros consecutivos en orden alfabético.

Variedad	Entero
BARBUNYA	0
BOMBAY	1
CALI	2
DERMASON	3
HOROZ	4
SEKER	5
SIRA	6

El `MLPClassifier` de `scikit-learn` requiere etiquetas numéricas. El objeto “`le`” se retorna para poder recuperar los nombres originales de las variedades a partir de las predicciones del modelo usando `le.inverse_transform(predicciones)`.

Resultado obtenido:

- **X.shape - (13,543, 16)**
- **y.shape - (13,543)**
- **7 clases codificadas como enteros 0–6**

Módulo `split_data.py`

Este módulo divide los datos preprocesados en tres conjuntos con roles distintos dentro del pipeline. La partición es reproducible gracias a una semilla fija y mantiene proporciones de clase de forma explícita mediante estratificación.

RANDOM_STATE = 17

La semilla 17 garantiza que cada ejecución del pipeline produzca exactamente la misma división, condición necesaria para que los experimentos sean reproducibles y comparables.

Función split_data(X, y)

```
def split_data(X, y):
    """
    Divide X e y en train (70%), validation (15%) y test (15%).
    Usa estratificación para mantener la proporción de clases en cada partición.

    El conjunto de validation se usa exclusivamente como función de fitness
    durante la optimización híbrida GWO-GA. El conjunto de test permanece
    sellado hasta la evaluación final.

    Returns:
        X_train, X_val, X_test, y_train, y_val, y_test
    """
    X_train, X_holdout, y_train, y_holdout = train_test_split(
        X, y,
        test_size=0.30,
        random_state=RANDOM_STATE,
        stratify=y
    )

    X_val, X_test, y_val, y_test = train_test_split(
        X_holdout, y_holdout,
        test_size=0.50,
        random_state=RANDOM_STATE,
        stratify=y_holdout
    )

    return X_train, X_val, X_test, y_train, y_val, y_test
```

La división se realiza en dos pasos porque `train_test_split` solo produce subconjuntos por llamada.

Paso 1 — Primera división (70% / 30%)

```
X_train, X_holdout, y_train, y_holdout = train_test_split(
    X, y,
    test_size=0.30,
    stratify=y
)
```

Se separa el 70% destinado al entrenamiento. El 30% restante se denomina holdout (datos reservados) y no se usa directamente; es un conjunto intermedio que se divide en el segundo paso siguiente.

Paso 2 — Segunda división del holdout (15% / 15%)

```
X_val, X_test, y_val, y_test = train_test_split(
    X_holdout, y_holdout, test_size=0.50,
    stratify=y_holdout
)
```

El 30% reservado se parte exactamente a la mitad, produciendo dos subconjuntos de igual tamaño, cada uno equivalente al 15% del dataset original.

Resultado obtenido:

Conjunto	Muestras	Porcentaje	Rol
X_train	9,480	70%	Entrenamiento del modelo
X_val	2,031	15%	Función de fitness del optimizador GWO-GA
X_test	2,032	15%	Evaluación final — permanece sellado

Por qué estratificación (stratify=y):

```
stratify=y
stratify=y_holdout
```

Sin estratificación, la asignación aleatoria podría distribuir las clases de forma desigual entre particiones, especialmente crítico para “BOMBAY” que representa solo el 3.8% del dataset (522 muestras). Con estratificación, cada subconjunto mantiene exactamente las mismas proporciones de clase que el dataset original, garantizando que el modelo vea ejemplos representativos de todas las variantes durante el entrenamiento y que las métricas de validación y test sean confiables.

1. La sintaxis de Python (desempaquetado)

En Python, una función puede regresar múltiples valores al mismo tiempo. La función `train_test_split` siempre te devuelve exactamente 4 pedazos de datos cuando le das 2. La sintaxis:

```
a, b, c, d = funcion()
```

Se llama desempaqueado (unpacking). Python toma el primer pedazo y lo guarda en `X_train`, el segundo en `X_holdout`, y así sucesivamente, todo en una sola línea elegante.

2. ¿Por qué "X" mayúscula y "y" minúscula?

`X` (mayúscula): Representa una matriz matemática. Tus datos tienen 16 características (área, perímetro, etc.) y miles de filas. Si graficaras esto, no sería un plano 2D ni un espacio 3D, sería un hiperespacio de 16 dimensiones. Como los matemáticos representan las matrices con letras mayúsculas, en IA siempre usamos la `X` mayúscula para las variables independientes (las características del frijol).

`y` (minúscula): Representa un vector. Es una simple lista o columna de números (en tu caso, un solo número que representa si es SIRA, DERMASON, etc.). En matemáticas, los vectores columna siempre se escriben con minúscula, por eso usamos `y` minúscula para la variable dependiente (la respuesta correcta).

¿Qué es el "holdout"?

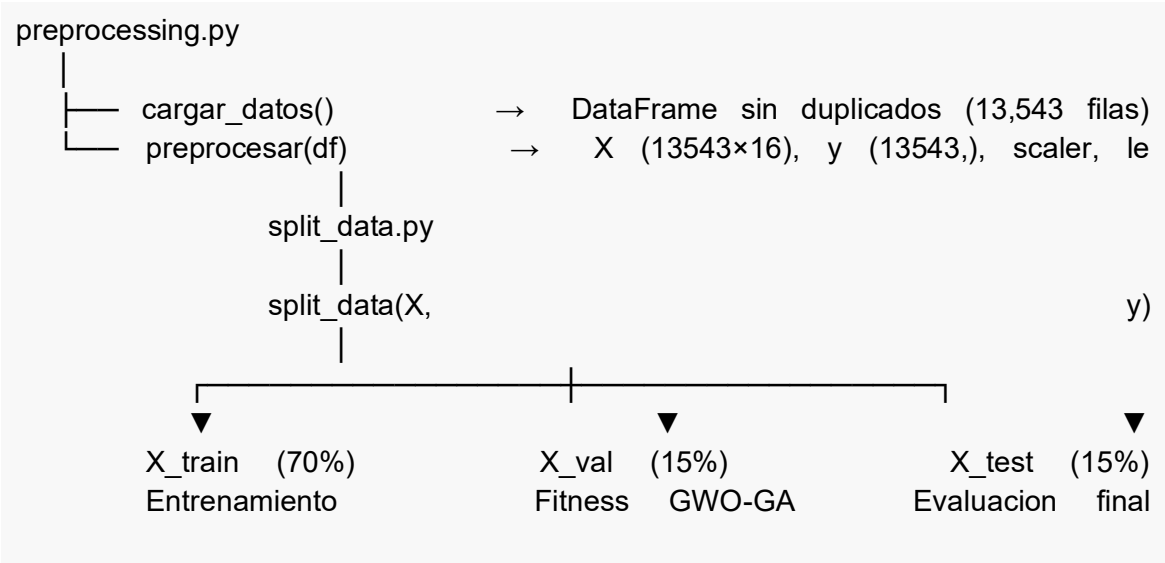
Como `train_test_split` originalmente solo divide en 2 partes (Train y Test), cuando necesitamos 3 partes (Train, Validation y Test), hacemos un pequeño truco:

Cortamos el 70% para train. El 30% restante lo llamamos temporalmente holdout, o "lo que queda".

Luego agarramos ese holdout y lo partimos por la mitad (`test_size=0.50`), lo que nos genera el 15% para validation y el 15% para test.

- `X` = Una matriz donde cada fila es un frijol y cada columna es una de las 16 features (Área, Perímetro, etc.). Si tienes 10,000 frijoles, tu matriz `X` es de 10,000 x 16.
- `y` = Un arreglo (vector) largo de 10,000 elementos, donde cada elemento es un solo número (del 0 al 6) que representa a cuál de las 7 clases pertenece ese frijol específico.

Flujo combinado en el pipeline principal



El scaler y `le` retornados por `preprocesar` se conservan en `main.py` para ser aplicados sobre cualquier dato nuevo sin recalculer los parámetros de normalización.